VLM Hallucination Mitigation: System Diagrams & Methodology Workflows

Overview

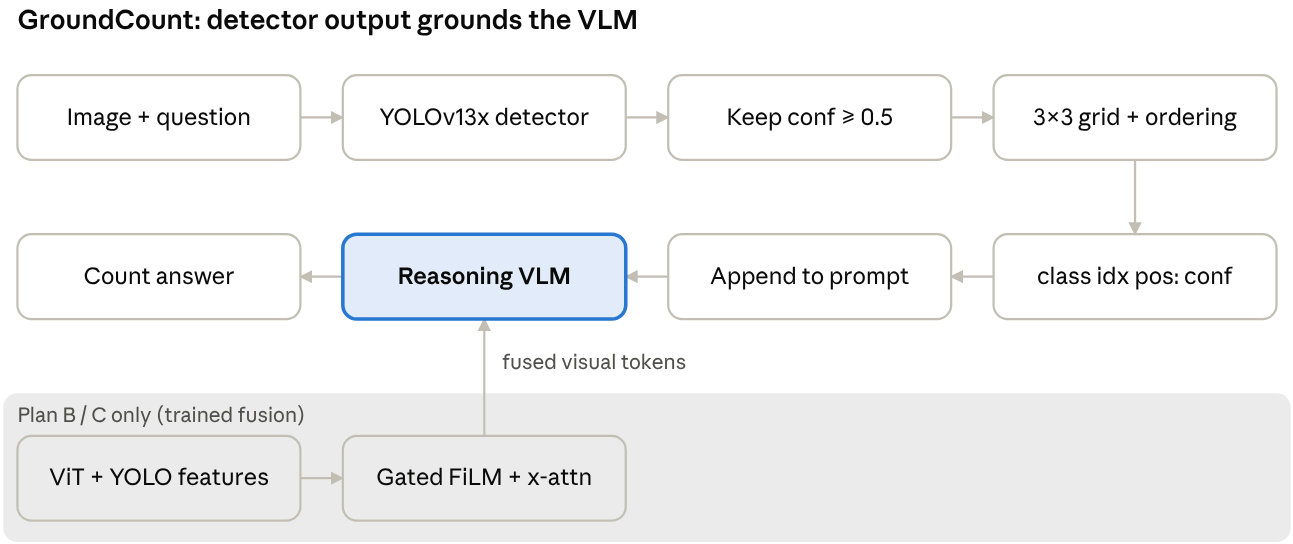
Method	Venue	Hallucination targeted	Intervention point	Needs model internals?	Headline result
GroundCount	arXiv 2026	Counting	Prompt (YOLO detections as text)	No (Plan A)	Ovis2.5-2B counting 74.7% → 81.3%, 22% faster
EnAR	CVPR 2026	Counterfactual (image contradicts world knowledge)	Visual tokens + contrastive decoding	Yes (encoder attention, logits)	+10.82% on VLMBias, +6.9% avg on POPE
Low-Rank method	arXiv 2025	Driving-scene captions	Output selection across captions	No	87% hallucination-free selection, 0.84 s per scene
BetterCheck	ITSC 2025	Made-up or missed traffic agents	Output filtering by self-check	No	GPT-4o 99.6% sentence correctness on Waymo subset
Visual Evidence Prompting	ACL 2025	Object, attribute, relation	Prompt (detector + scene-graph text)	No	LLaVA-1.5-7B POPE-Adv 80.23% → 87.43%
Activation Steering Decoding	ACL 2025	Object	Hidden states + contrastive decoding	Yes (hidden states, logits)	LLaVA-1.5-7B $CHAIR_S$ 51.0 → 40.0
ICT	CVPR 2025	Object (image- and object-level)	Attention-head activations (forward pass)	Yes (weights, head outputs)	LLaVA-v1.5 POPE-COCO-Random F1 81.33 → 90.03, ~no extra latency
OPERA	CVPR 2024	Object (long captions)	Beam-search scores + rollback	Yes (self-attention, logits)	LLaVA-v1.5 POPE-COCO-Random F1 81.33 → 88.81 (ICT re-run)
VCD	CVPR 2024	Object (language prior, co-occurrence)	Logits (clean vs noised image)	Yes (logits)	Up to +7.4 POPE F1 across LLaVA-1.5, InstructBLIP, Qwen-VL

Method	Venue	Hallucination targeted	Intervention point	Needs model internals?	Headline result
HELTA	IEEE T-ITS 2026	Object hallucinations + missed traffic objects	Output text (VLM voting + detector enrichment)	No	Mini-InternVL-4B POPE F1 +12.58, mPLUG-Owl3 +4.28

Numbers are each paper's own best reported setting and are not comparable across rows (different models and benchmarks).

1. GroundCount — object-detector grounding for counting

GroundCount runs a YOLOv13x detector first and hands its detections to the VLM as structured text; the training-free Plan A lifts Ovis2.5-2B counting accuracy on the PhD benchmark from 74.7% to 81.3% and cuts inference time 22% (10.0 s → 7.8 s). Paper: GroundCount (arXiv 2603.10978).



GroundCount system · Plan A prompt path + Plan B/C fusion lane

Plan A is the top two rows (no training). Plans B and C add a trained fusion network that injects detector features into the VLM's visual tokens before decoding.

Methodology workflow

1. **Detect.** Run YOLOv13x on the image to get class labels, bounding boxes and confidence scores (~0.1 s).

2. **Filter.** Drop detections below confidence 0.5. Lowering to 0.3 cost 1.3–16.9 pp accuracy: precision matters more than recall.
3. **Encode position.** Map each box centre to a 3×3 grid cell (upper/middle/lower × left/center/right).
4. **Sequence.** Order objects left-to-right, then lower-to-upper, and index repeats of a class.
5. **Format and append.** Write one line per object as `[class] [index] [position]: [confidence]` and append the list to the user question.
6. **Reason and answer.** The VLM reasons over image + detection list; grounding stops the endless re-checking loops that waste tokens.
7. **Plan B (optional, trained).** Per ViT patch p_i, fuse the matching CNN feature c_i and a pooled global feature g through two branches, FiLM ($\gamma_i \odot p_i + \beta_i$) and cross-attention, mixed by a learned gate α_i, then an information bottleneck. Trained on COCO train2017 to emit `[class] [index] in [position]`, AdamW, lr 2×10^{-5}, up to 40k steps.
8. **Plan C.** Plan B.4 fusion plus the Plan A prompt.

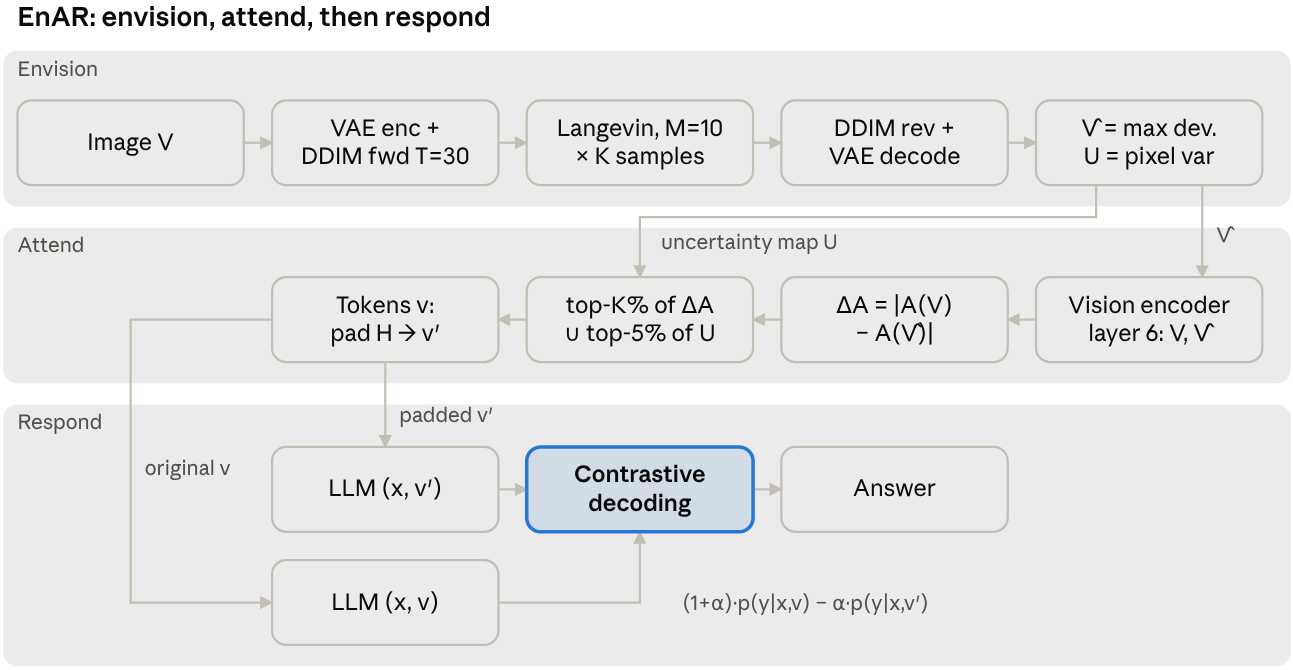
Variant results (Ovis2.5-2B, PhD counting subset)

Variant	What is trained	Accuracy (%)	Time (s)
Baseline	nothing	74.7	10.0
Plan A — ODM prompt	nothing	81.3	7.8
Plan B.1	fusion only	75.2	17.8
Plan B.2	fusion + VLM + ODM	72.7	14.6
Plan B.3	fusion + language decoder	71.4	4.3
Plan B.4	fusion + VLM + ODM + decoder	78.0	7.7
Plan C — A + B.4	as B.4	78.2	4.8
YOLOv13x alone	—	72.8	0.1

Plan A gains 6.2–7.5 pp on four of five VLMs; InternVL3.5-1B drops (64.0% → 62.5%), which the authors tie to its iterative reflection clashing with structured prompts.

2. EnAR — Envision, Attend, Then Respond (CVPR 2026)

EnAR is a training-free plug-in: a diffusion model "envisions" a prior-consistent version of the image, the LVLM's own vision encoder compares attention on the two images to find counterfactual tokens, and contrastive decoding against a padded copy suppresses the language prior. The paper reports a 10.82% gain on VLMBias and a 6.9% average gain on POPE across InternVL3.5-8B, Qwen2.5-VL-7B and LLaVA-v1.5-7B. Source: CVPR 2026 paper (Sec. 3, Fig. 2–3, Algorithm 1), code.



EnAR system · 3 stages, 2 decoding branches

Both the question x and a visual token sequence enter the LLM twice: once with the original tokens v and once with v′, where the counterfactual tokens (e.g. the five-legged llama's extra leg) are replaced by ⟨pad⟩. The v′ branch speaks for the language prior, so subtracting it pushes the answer toward the pixels.

Methodology workflow

1. **Envision — invert to a noisy latent.** Encode the image with the Stable Diffusion v1.5 VAE and run deterministic DDIM forward to step T = 30 of T_{max} = 50: z_T = DDIMForward(Encoder(V), T). DDIM is used instead of DDPM because it is invertible and controllable.

2. **Envision — latent perturbation.** For each of K samples, copy z_T and run M = 10 steps of annealed Langevin dynamics toward higher likelihood under the image prior: $\hat{z}_T \leftarrow \hat{z}_T - \eta \cdot \varepsilon_\theta(\hat{z}_T, T)/\sqrt{(1-\bar{\alpha}_T)} + \sqrt{(2\eta\tau)} \cdot \xi$, with $\xi \sim N(0, I)$. The score $G = \nabla \log p(z_T)$ is estimated with Tweedie's formula as $-\varepsilon_\theta(z_T, T)/\sqrt{(1-\bar{\alpha}_T)}$; η anneals from 10^{-2} to 10^{-4} and $\tau = 0.1$.
3. **Envision — decode.** $V^{(k)} = \text{Decoder}(\text{DDIMReverse}(\hat{z}_T^{(k)}, T))$ for each sample. Plausible regions already sit in high-likelihood space and barely change; implausible ones are "normalised".
4. **Envision — uncertainty estimation.** $U_{ij} = \text{Var}(\{V^{(k)}_{ij}\})$ over the K impressions; the final impression $\hat{V}$ is the sample that deviates most from the input.
5. **Attend — contrastive attention.** Pass V and $\hat{V}$ through the LVLM's own vision encoder and read layer L = 6: the ⟨cls⟩-to-patch attention, or, for encoders without ⟨cls⟩ (Qwen-VL), each token's summed incoming attention averaged over heads. Compute $\Delta A = |\text{Attn}^{(L)}(V) - \text{Attn}^{(L)}(\hat{V})|$.
6. **Attend — localise.** H_{attn} = top-K% tokens by ΔA; H_{unc} = top-5% most uncertain positions of U mapped to token indices; $H = H_{attn} \cup H_{unc}$. The padding-token ratio is fixed at 10%.
7. **Attend — padded input.** In the visual embedding sequence v, replace the tokens indexed by H with the model's built-in ⟨pad⟩ token to get v′; v stays intact.
8. **Respond — contrastive decoding.** Run the LLM on (x, v) and (x, v′) and decode with $p(y|x, v, v') = (1+\alpha) \cdot p(y|x, v) - \alpha \cdot p(y|x, v')$; α and the other decoding settings follow VCD.
9. **No training.** All models (diffusion prior, LVLM) are frozen; the same settings (layer 6, 10% padding) are used for every backbone.

Reported results (overall accuracy %, VLMBias)

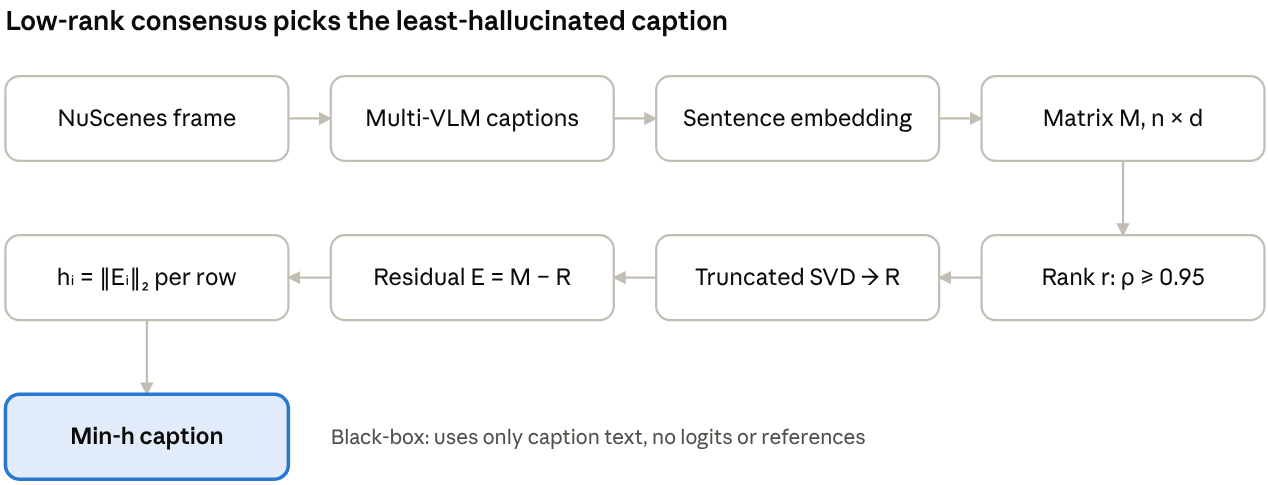
Backbone	Regular	Best baseline	EnAR
InternVL3.5-8B	19.83	23.76 (VCD)	31.36
Qwen2.5-VL-7B	22.63	24.78 (VCD)	28.02
LLaVA-v1.5-7B	16.92	19.18 (VCD)	22.20

Ablation (VLMBias overall, InternVL3.5-8B): 31.36 full, 29.03 without the uncertainty map, 25.80 without the visual impression (tokens chosen from U of the original image only), 19.83 regular decoding. Layers near the 6th and a 10% padding ratio gave the best average

across VLMBias, POPE and HallusionBench. Every method, EnAR included, scores 0 on VLMBias Chess Pieces. Other benchmarks: PhD (counting), WHOOPS, HallusionBench and POPE, where LLaVA-1.5-7B F1 rises 8.1, 8.0 and 6.9 points on Random, Popular and Adversarial. Cost: K diffusion perturbations plus two encoder passes and two LLM passes per query.

3. Low-Rank Method for VLM Hallucination Mitigation in Autonomous Driving

The method treats several VLM captions of one driving frame as a matrix of sentence embeddings, keeps the low-rank consensus, and picks the caption with the smallest residual; on NuScenes it selects a hallucination-free caption 87% of the time in 0.84 s, 51–67% faster than multi-agent debate. Paper: [arXiv 2511.06496].



Low-rank caption selection · generate, embed, decompose, score, pick

Consensus lives in the few dominant singular directions; a caption that adds made-up objects, colours or positions sits outside them and leaves a large residual.

Methodology workflow

1. **Generate candidates.** Prompt one or more VLMs (DeepSeek-VL2-tiny, LLaVA-1.5-7B, Qwen2-VL-7B) with a traffic-focused prompt; temperature 0.7, max 150 tokens, 10 captions per model, run in parallel.
2. **Embed.** Encode each caption with a pretrained sentence transformer, $e_i = \mathrm{Enc}(c_i) \in \mathbb{R}^d$ ($d = 1024$).
3. **Stack.** Rows form $M \in \mathbb{R}^{n \times d}$.

4. **Choose rank.** Full SVD, then the smallest r whose cumulative variance ratio $\rho_k \geq 0.95$.
5. **Decompose.** Truncated SVD $R = U_r \Sigma_r V_r^T$ (chosen over RPCA because paraphrase noise is dense, not sparse); residual $E = M - R$.
6. **Score.** $h_i = \|E_{i,:}\|_2$ for each caption.
7. **Select.** Sort ascending and output the lowest-h caption.
8. **Evaluate (offline only).** GPT-4o labels each sentence against NuScenes 3D boxes and scene metadata, with manual checks; metrics are top-1 selection accuracy and Spearman correlation between h_i and the labelled score.

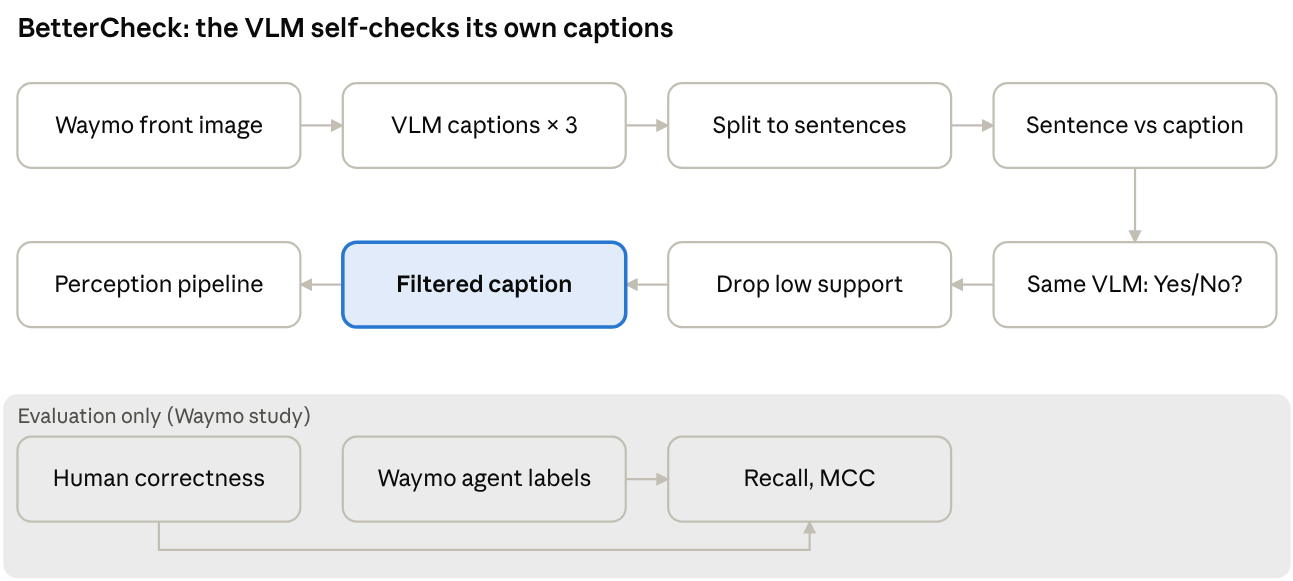
Reported results (300 NuScenes keyframes, downscaled to ~400×300)

Method	Selection accuracy	Time per scene (s)
Heterogeneous multi-agent low-rank	0.87	0.84
Debate led by DeepSeek	0.82	1.80
Debate led by Qwen	0.81	1.72
Debate led by LLaVA	0.79	2.56
Homogeneous low-rank (LLaVA)	0.81	—
Single model, no filtering (LLaVA)	0.80	—

Pooling all three models raises the share of scenes with positive Spearman correlation to 92% (mean 0.71). Limitation: if every model hallucinates the same object, consensus cannot catch it.

4. BetterCheck — safeguarding VLMs for automotive perception (ITSC 2025)

BetterCheck adapts SelfCheckGPT to driving images: the VLM captions a frame three times, then the same model judges whether each sentence is supported by its other captions, and unsupported sentences are dropped as likely hallucinations. It is as much an evaluation study (GPT-4o, LLaVA, MiniCPM-V on 500 Waymo images) as a method. Paper: arXiv 2507.17722.



BetterCheck · self-check guardrail + evaluation lane

The top two rows run at inference with no training and no model internals; the shaded lane is how the authors measured hallucinations (made-up agents) and overlooks (missed agents).

Methodology workflow

1. **Curate data.** 500 front-camera images (1920×1280) from 20 Waymo Open Dataset scenarios, every tenth frame, 25 hand-picked per scenario across weather, day/night and neighbourhoods.
2. **Caption, best of three.** Feed each image 3 times to the VLM (via Ollama) with a template prompt: one short sentence per object type, `There are [object].`, with emphasis on people.
3. **Decompose.** Split each caption R_i into sentences $s_1 \ldots s_n$ (sentences over 50 characters discarded).
4. **Self-check.** Pair every sentence with each of the other captions and ask the same VLM: `Context: … Sentence: … Is the sentence supported by the context above? Answer Yes or No`.
5. **Filter.** Remove sentences with low consistency across repetitions; the rest is the safeguarded caption passed to perception.
6. **Annotate (evaluation).** Humans mark each sentence correct/incorrect (15% overlap, Cohen's kappa 50–80%) and cross-check sentences against Waymo labels (vehicles, pedestrians, signs, cyclists) to count overlooked agents.

7. **Analyse.** Precision, recall, F1, MCC; recall is the key safety metric because it captures overlooked road users.

Reported results

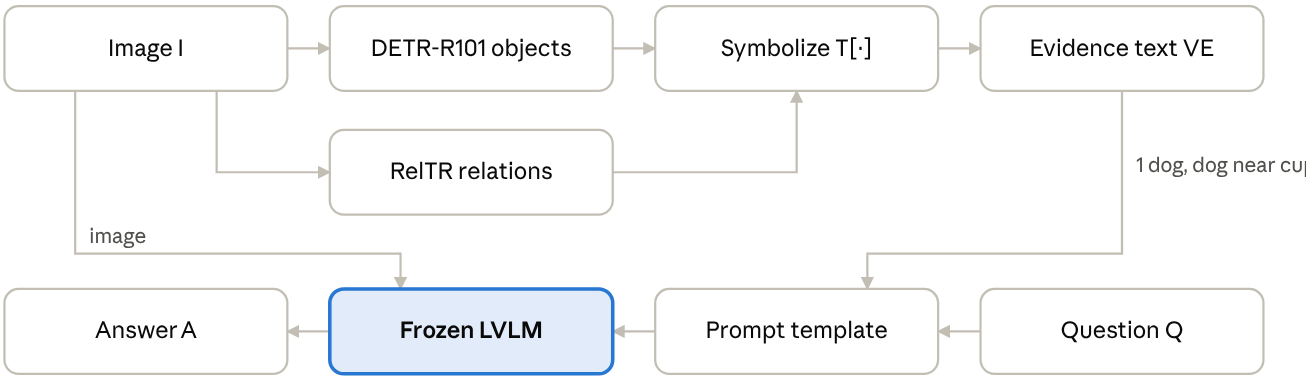
Model	Sentence correctness	Caption correctness	Recall vs Waymo agents	Self-check MCC
GPT-4o	99.6%	97.1%	78.04%	0.075
MiniCPM-V	94.8%	88%	25.56%	0.370
LLaVA	85.6%	71.9%	56.41%	0.138

Low self-check MCC means the checker agrees with humans only weakly; the authors also note current VLMs are still too slow or cloud-dependent for in-vehicle deployment.

5. Visual Evidence Prompting (ACL 2025)

VEP runs small expert models (an object detector and a scene-graph generator) on the image, turns their output into a sentence of evidence, and prepends it to the question; with no training and no access to weights it lifts LLaVA-1.5-7B on POPE-Adversarial from 80.23% to 87.43% and also helps API models such as GPT-4V and Claude 3. Paper: Li et al., ACL 2025.

Visual evidence prompting: small experts feed the LVLM



VEP system · 2 small visual experts, 1 frozen LVLM

The LVLM still sees the image; evidence only steers where it looks. Swapping in a random image drops LLaVA-1.6-7B from 89.43% to 69.30%, so the model is not just reading the text.

Methodology workflow

1. **Extract objects.** Run DETR-ResNet-101 on the image; merge same-class detections into counts, e.g. `3 dogs, 1 cat`.
2. **Extract relations.** Run RelTR scene-graph generation; write each ⟨subject, relation, object⟩ triplet as `subject relation object`, joined by commas.
3. **Symbolize.** VE = T[f_SVM(I)], the transform from structured detector output into natural language.
4. **Prompt.** Build `You can see {evidence} in the image. {question}` and feed it with the original image: A = f_LVLM(I, Q, VE).
5. **Answer.** The frozen LVLM (open or API) answers; no logits, weights or fine-tuning are needed, and VEP stacks with VCD or LRV.

Reported results (POPE COCO-Adversarial accuracy %)

Model	Baseline	+ Visual evidence
Claude 3 (API)	75.40	87.50
MiniGPT-4-v2	75.33	83.17
LLaVA-1.5-7B	80.23	87.43
Gemini 1.5 Pro (API)	82.43	87.32
GPT-4V (API)	82.21	86.41
LLaVA-1.6-7B	84.93	89.43

Detector quality matters more than recall: with LLaVA-1.5-7B, accuracy rises from 84.13% (YOLOS-tiny, 28.7 mAP) to 88.00% (DINO-Swin, 58.0 mAP), then saturates. The main remaining error (58%) is the LVLM ignoring correct evidence.

6. Activation Steering Decoding (ACL 2025)

ASD learns a "hallucination direction" in the decoder's hidden states from a small labelled calibration set, then at every step runs two passes steered in opposite directions and contrasts their logits; on LLaVA-1.5-7B it cuts $CHAIR_S$ from 51.0 to 40.0 and raises POPE-COCO accuracy from 85.13% to 88.01%, for 10–20% extra latency. Paper: Su et al., ACL 2025.

ASD system · offline calibration + dual-pass contrastive decoding

The offline band runs once per model; the online band runs every token, with both steered passes batched together (batch size 2).

Methodology workflow

1. **Generate calibration captions.** Prompt the base LVLM with `Please describe the image in detail.` on 1,000 MSCOCO training images.
2. **Label tokens.** Split each caption into sentences with NLTK; match the 80 COCO classes plus synonyms; a sentence naming any object absent from the ground truth marks all its tokens as hallucinated.
3. **Collect activations.** Store hidden states z_l for every token at every layer. Linear probes reach 82.49% accuracy in middle layers, showing hallucination is linearly separable.
4. **Build steering vector.** Per layer, v_l = mean activation of factual tokens − mean activation of hallucinated tokens.
5. **Steer two ways.** At inference, π^+ adds $+\lambda^+ v_l$ and π^- adds $-\lambda^- v_l$ to the hidden states ($z_l + \lambda v_l$); best setting $\lambda^+ = 0.2$, $\lambda^- = 0.4$.
6. **Contrast.** $\text{logit_ASD} = (1+\alpha)\cdot\text{logit}^+ - \alpha\cdot\text{logit}^-$, then pick the next token. The contrast lets small λ values work, so hidden-state semantics are not distorted.

Reported results (LLaVA-1.5-7B)

Benchmark	Baseline	VCD	ASD
$CHAIR_S$ (lower is better)	51.0	47.8	40.0
$CHAIR_I$ (lower is better)	14.7	14.1	11.3
POPE-COCO accuracy, greedy (%)	85.13	85.16	88.01

Benchmark	Baseline	VCD	ASD
POPE-GQA accuracy, greedy (%)	76.60	76.49	83.49
MME total	1810.70	1800.41	1827.21

Even 100 calibration images get most of the gain (POPE 87.72%). Limitation: the vector comes from COCO object labels, so it targets object hallucination, not attributes or relations.

8. ICT — Image-Object Cross-Level Trusted Intervention (CVPR 2025)

ICT precomputes "trusted minus untrusted" activation shifts for selected attention heads, one set for the whole image and one for the queried object, then adds them during a normal forward pass; it lifts LLaVA-v1.5 POPE-COCO-Random F1 from 81.33 to 90.03 with essentially no extra latency. Paper: arXiv 2411.15268.

ICT system · image- and object-level shift vectors + one intervened forward pass

Unlike contrastive decoding, nothing is subtracted at the logits, so useful language priors (e.g. knowing who a famous person is) survive; the model is only pushed to attend more to the image.

Methodology workflow

1. **Build calibration pairs.** Take 1,500 POPE COCO-Random questions answered "Yes" ("Is there a/an [object] in the image?"), extract the object O_i, and rewrite the prompt as q = "What is it in the image?" with answer q + O_i.
2. **Image-level untrusted input.** Add forward-diffusion Gaussian noise to the whole image to get V'_i.
3. **Object-level untrusted input.** Locate O_i with Grounding DINO and add the same noise only inside that box to get V''_i.
4. **Collect activations.** Run trusted (V_i) and untrusted inputs; record every head's attention output at the last token for all N×L heads (1,024 heads in LLaVA-v1.5).
5. **Shift vectors.** Per head, $S^{(l)}_n$ = mean(A − A′) for image level and $S^{(l)}$_obj,n = mean(A − A″) for object level.

6. **Select heads.** Train an SVM per head (2-fold cross-validation) to separate trusted from untrusted activations; keep the top-K heads by accuracy for each level. Image-level heads cluster in early layers (e.g. 4, 10), object-level heads in later layers (e.g. 20, 22).
7. **Intervene at inference.** In each layer, $H^{(l+1)} = H^{(l)} + \Sigma_n (Attn_n(H^{(l)}) + \mathbb{1}_img \cdot \alpha S_n + \mathbb{1}_obj \cdot \beta S_obj,n) \cdot W_o$, applied at the attention output projection.
8. **Tune.** $\alpha = \beta \in \{8, 16, 24, 32\}$, $K \in \{32, 64, 128, 256\}$ by grid search; greedy decoding.

Reported results (LLaVA-v1.5, POPE COCO, F1)

Setting	Regular	VCD	OPERA	ICT
Random	81.33	87.16	88.81	90.03
Popular	80.06	85.06	86.62	87.60
Adversarial	77.57	81.33	81.38	83.74

Latency for 20 tokens on an H800: 405.3 ms regular, 415.9 ms ICT, 988.9 ms VCD, 1,371.7 ms OPERA. Limitation: needs model weights, so it cannot be used with closed API models.

9. OPERA — Over-Trust Penalty and Retrospection-Allocation (CVPR 2024)

OPERA is a training-free beam-search decoder: it detects "knowledge aggregation" columns in the self-attention map (the model over-trusting a few summary tokens such as a full stop instead of the image), penalises beams that show the pattern, and rolls back to the summary token when every beam is already affected. Paper: arXiv 2311.17911, code.

OPERA decoding loop · penalty on beam scores + retrospective rollback

The penalty acts on beam scores, so it steers which continuation survives; the rollback handles the lag, because the aggregation column only becomes visible several tokens after the hallucination has started.

Methodology workflow

1. **Observation that motivates it.** Hallucinations often begin within the first 10 tokens after a columnar attention pattern on a low-information token; CHAIR rises as more such summary tokens appear in a long caption.
2. **Beam search base.** Decode with N_beam beams; each beam proposes its top candidate next tokens.
3. **Local window.** For each candidate, take the causal self-attention weights among the last k generated tokens, $W^k_{t-1} = \{w^i\}$, excluding image and prompt tokens ($t - k \geqslant N + M$). Use the maximum over heads and renormalise.
4. **Over-trust score.** Scale the window values up, multiply down each column (column-wise product) and take the maximum column score as the pattern intensity φ. A large φ means many recent tokens lean on one summary token.
5. **Over-trust logit penalty.** Subtract $\alpha \cdot \varphi$ from the candidate's log-probability before beam scoring, so beams showing the pattern drop in priority.
6. **Retrospection check.** Track where the maximum column falls for recent tokens; if the same location (summary token) recurs up to a threshold, trigger rollback.
7. **Allocation.** Roll decoding back to that summary token and re-select the next token from the remaining candidates, excluding the one chosen before, then continue beam search.
8. **No training or extra data.** Everything uses the model's own attention and logits at inference; the cost is several times slower decoding.

Reported results (LLaVA-v1.5, POPE COCO F1, as re-run in the ICT paper)

Setting	Regular	VCD	OPERA
Random	81.33	87.16	88.81
Popular	80.06	85.06	86.62
Adversarial	77.57	81.33	81.38

The original paper evaluates InstructBLIP, MiniGPT-4, LLaVA-1.5 and Shikra on CHAIR, POPE and GPT-4/GPT-4V judgments. The ICT paper measures OPERA at about 3.4–3.6× the latency of regular decoding on LLaVA-v1.5. OPERA needs attention weights, so it does not work through closed APIs.

10. VCD — Visual Contrastive Decoding (CVPR 2024)

VCD runs the LVLM twice per token, once on the real image and once on a noise-distorted copy, and subtracts the distorted logits so tokens driven by language priors and co-occurrence bias lose out; with no training it improves POPE F1 by up to 7.4 points on LLaVA-1.5, InstructBLIP and Qwen-VL. Paper: arXiv 2311.16922, code.

VCD system · 2 forward passes, contrast, plausibility mask

VCD is the template that EnAR, and the contrastive step in ASD, build on: EnAR swaps the whole-image noise for targeted ⟨pad⟩ masking of counterfactual tokens, and ASD swaps the distorted image for a negatively steered pass.

Methodology workflow

1. **Motivation.** More visual uncertainty pushes the model toward language priors (a black banana becomes "yellow") and toward frequent or co-occurring COCO objects.
2. **Distort the image.** Add Gaussian noise following the forward diffusion process, $q(v_t \mid v_{t-1}) = N(\sqrt{(1-\gamma)} \cdot v_{t-1}, \gamma I)$, for T steps to get v' ($\gamma = 0.1$; $T = 999$ for POPE, 500 for MME and LLaVA-Bench).
3. **Two forward passes.** At each step compute $\text{logit}_\theta(y \mid v, x, y<t)$ and $\text{logit}_\theta(y \mid v', x, y<t)$.
4. **Contrast.** $p_vcd = \text{softmax}[(1+\alpha) \cdot \text{logit}(y \mid v, x) - \alpha \cdot \text{logit}(y \mid v', x)]$, $\alpha = 1$ ($\alpha = 0$ is ordinary decoding).
5. **Adaptive plausibility constraint.** Keep only $V_head = \{y_t : p(y_t \mid v, x) \geq \beta \cdot \max_w\, p(w \mid v, x)\}$, $\beta = 0.1$, and set $p_vcd = 0$ elsewhere, so the subtraction cannot promote implausible tokens.
6. **Sample.** Draw y_t from the masked p_vcd with any sampler (direct, top-p, top-k, greedy or beam) and repeat.

Reported results (POPE MSCOCO F1, direct sampling)

Model	Setting	Regular	VCD
LLaVA-1.5	Random	81.33	87.16
LLaVA-1.5	Adversarial	77.57	81.33
Qwen-VL	Random	82.67	87.81

Model	Setting	Regular	VCD
InstructBLIP	Random	80.41	83.68

On the MME hallucination subset LLaVA-1.5 rises from 565.33 to 604.66. Cost: two forward passes per token (about 2.1–2.4× latency in the ICT paper's timing).

11. HELTA — Hallucination Elimination and Text Annotation (IEEE T-ITS 2026)

HELTA is a training-free, post-hoc annotation pipeline for traffic scenes: it deletes objects in an LVLM description that three off-the-shelf VLMs vote as absent, then adds one-sentence descriptions of traffic objects the LVLM missed; it lifts POPE F1 by 12.58 points on Mini-InternVL-4B and 4.28 on mPLUG-Owl3, and was used to build the CODA_desc and nuScenes_desc datasets. Source: Fan et al., IEEE Trans. Intell. Transp. Syst., vol. 27, no. 1, Jan. 2026 (the uploaded paper, Sec. III, Fig. 3, Algorithms 1–2); code and datasets.

HELTA system · HaCCF text lane + critical-object image lane, 6 stages

The top lane alone is HaCCF (removes hallucinations, raises precision); the bottom lane restores recall by adding missed pedestrians, motorcycles, cones and the like. Together they form the full HELTA pipeline.

Methodology workflow

1. **Generate the initial description.** Any LVLM (LLaVA-1.5, mPLUG-Owl, MiniGPT-4, InternVL, …) describes the traffic image with "Please describe the images in detail".
2. **① Sentence split and key-entity extraction.** A pretrained spaCy model splits the text into sentences; Llama-3.1-8B extracts concrete entity words per sentence under few-shot rules: general categories, no abstract words (e.g. "environment"), singular form, one line separated by periods, "None" if empty.
3. **② Entity cross-checking (Algorithm 1).** Ask BLIP-2-FlanT5XXL and InstructBLIP-FlanT5XXL "Is there a {word} in the image? Please answer only with yes or no." Both no → drop; both yes → keep; disagreement → InstructBLIP-Vicuna-13B decides. Combined error rate is typically below 2% (about 10% for one model).

4. ③ **Hallucinatory text correction.** Llama-3.1-8B rewrites each sentence given Entity_1 (original) and Entity_2 (verified), removing descriptions of entities that failed; singular and plural count as the same, and identical lists return the sentence unchanged.
5. ④ **Critical-object identification (Algorithm 2).** RAM++ tags the image, restricted to 22 traffic classes (person, pedestrian, car, truck, bus, motorcycle, motorcyclist, bicycle, traffic light, traffic cone, traffic sign, stop sign, road, pavement, and others); Grounding-DINO-B must localise each tag with a box score $\geq \alpha = 0.35$.
6. ⑤ **Critical-object description.** Only objects not already in the verified entity list are described, by BLIP-2-FlanT5XXL with "Describe the {object} in the image with only one sentence."
7. ⑥ **Text integration.** Append the new sentences, prefixed "Some additional information includes:", to the corrected description; this refined pair is the dataset annotation.
8. **Dataset curation.** Manual screening kept 9,263 pairs for CODA_desc and 33,084 for nuScenes_desc.

Reported results (POPE on CODA, F1 %)

Model	Setting	Without HELTA	With HELTA
LLaVA-1.5-7B	Popular	69.87	73.39
LLaVA-1.5-13B	Adversarial	72.14	75.02
mPLUG-Owl2	Popular	68.21	70.94
MiniGPT-4	Random	43.36	56.92
Mini-InternVL-4B	Random	68.88	81.46

Ablation (LLaVA-1.5-7B, popular): HaCCF alone reaches the best precision (64.35%), RAM++ alone the best recall (94.15%), and full HELTA the best F1 (73.39%). Cost: about 20 s per image on one A800 (about 15 s with module-parallel execution). Failure cases include rain, rare objects such as manhole covers, and heavy occlusion.

12. Unified view — where each method intervenes

The ten methods split into black-box methods that work on any API model (prompting in, filtering out) and white-box methods that need attention, hidden states or logits; EnAR, ASD and VCD all end in the same contrastive-decoding step, just with different "negative"

branches.

Taxonomy · 6 methods placed on 5 pipeline stages

GroundCount Plan A, VEP and HELTA's enrichment lane share one idea (small expert model → text evidence → prompt); both papers find detector precision matters more than recall. Low-Rank and BetterCheck both exploit consistency across multiple generations, one via embedding geometry, the other via the model judging itself.